

Họ và tên: Nguyễn Đông Quân

Mã số Sinh viên: 24521438

Lớp: IT007.Q21.VMTN

HỆ ĐIỀU HÀNH
BÁO CÁO LAB 02
TÓM TẮT BÁO CÁO

1. CLASSWORK

	EX 1	EX 2	EX 3	EX 4	EX 5
Phương pháp	☐	☐	☐	☐	☐
Minh chứng	☐	☐	☐	☐	☐
Giải thích	☐	☐	☐	☐	☐

2. HOMEWORK

	EX 1	EX 2	EX 3	EX 4	EX 5	EX 6	EX 7
Method	☐	☐	☐	☐	☐	☐	☐
Proof	☐	☐	☐	☐	☐	☐	☐
Explanation	☐	☐	☐	☐	☐	☐	☐

3. BONUS

	EX 1	EX 2
Method	☐	☐
Proof	☐	☐
Explanation	☐	☐

Tự đánh giá: Đã hoàn thành toàn bộ bài thực hành LAB02 kèm phương pháp, minh chứng và giải thích đầy đủ.

**Lưu ý: Xuất file PDF, đặt tên theo cú pháp:*

<MSSV>_LABx.pdf

REPORT

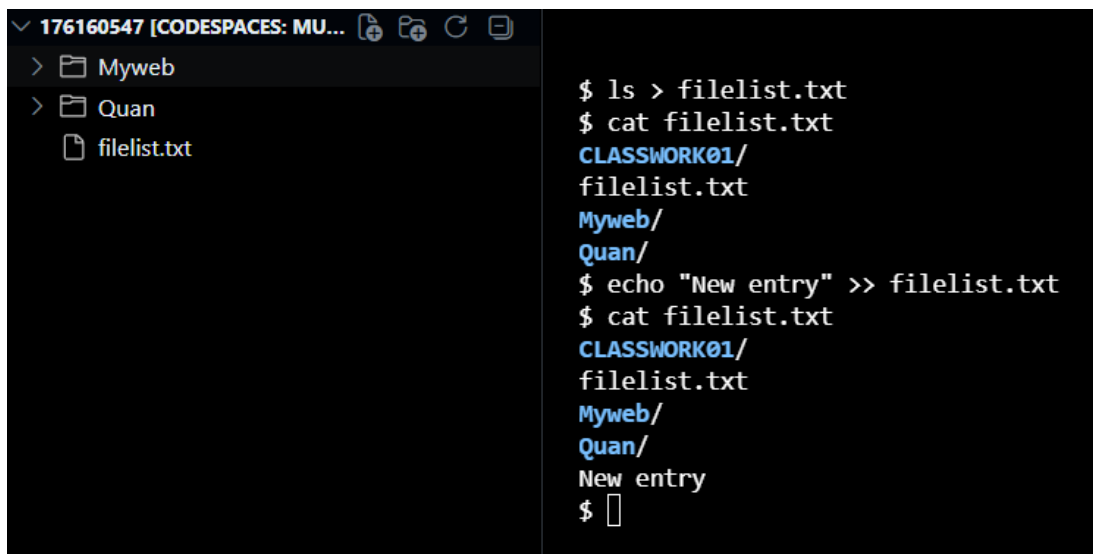
1. CLASSWORK

1.1 CLASSWORK 01:

***Phương pháp thực hiện:**

- Chuẩn bị dữ liệu: Tạo tệp tin văn bản rỗng hoặc có sẵn dữ liệu mẫu để làm đầu vào/đầu ra cho các lệnh.
- Thực thi kiểm thử: Lần lượt thực thi 8 ký tự chuyển hướng (>, >>, <, <<, 2>, 2>>, &>, &>>) kết hợp với các lệnh cơ bản như echo, ls, cat, sort, grep,...
- Đánh giá kết quả: Sử dụng lệnh cat để kiểm tra nội dung file sau khi chuyển hướng, từ đó đối chiếu với lý thuyết để giải thích ý nghĩa.

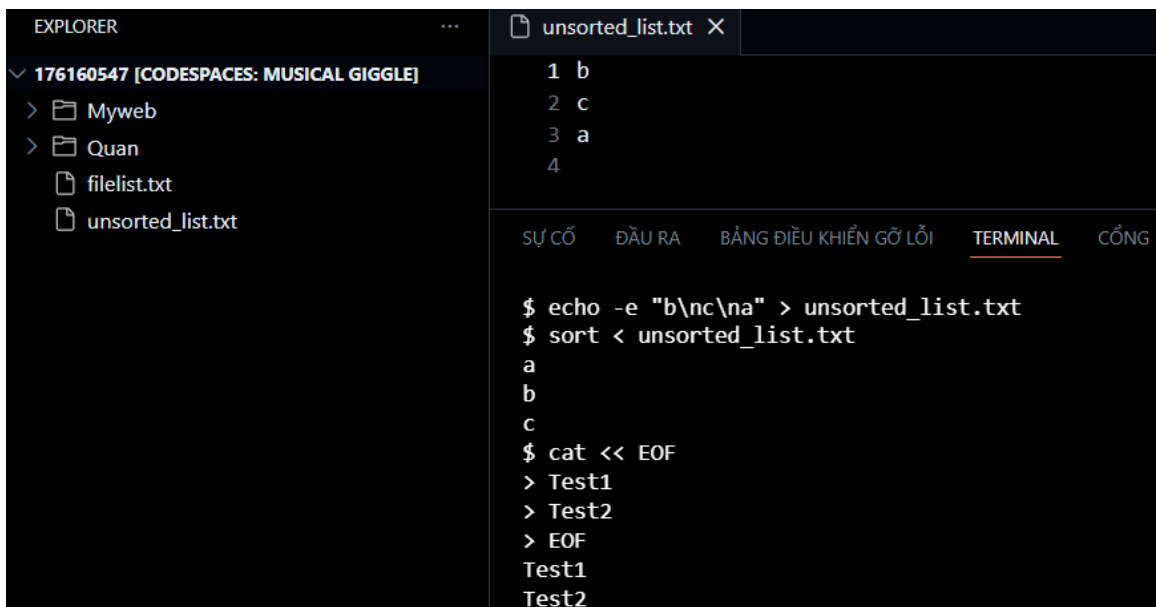
***Minh chứng và giải thích:**



```
176160547 [CODESPACES: MU...]  
> Myweb  
> Quan  
  filelist.txt  
  
$ ls > filelist.txt  
$ cat filelist.txt  
CLASSWORK01/  
filelist.txt  
Myweb/  
Quan/  
$ echo "New entry" >> filelist.txt  
$ cat filelist.txt  
CLASSWORK01/  
filelist.txt  
Myweb/  
Quan/  
New entry  
$
```

Thử nghiệm chuyển hướng đầu ra chuẩn (stdout) với > và >>

- > (**Chuyển hướng và ghi đè**): Chuyển hướng đầu ra (stdout) của lệnh ls vào tệp filelist.txt. Nếu tệp đã tồn tại, nội dung cũ sẽ bị ghi đè hoàn toàn bằng nội dung mới.
- >> (**Nối thêm đầu ra**): Nối thêm chuỗi "New entry" vào cuối tệp filelist.txt mà không làm mất dữ liệu hiện có. Nếu tệp chưa tồn tại, nó tự động tạo tệp mới.

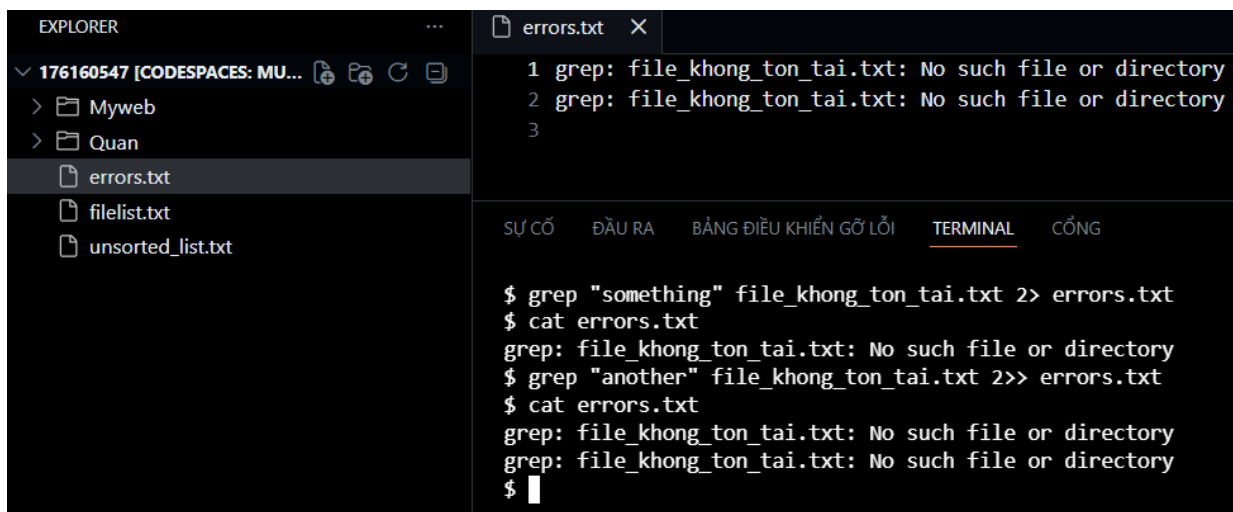


The screenshot shows a code editor interface with a file explorer on the left and a terminal on the right. The file explorer shows a directory structure with files 'filelist.txt' and 'unsorted_list.txt'. The terminal shows the following commands and output:

```
$ echo -e "b\nc\na" > unsorted_list.txt
$ sort < unsorted_list.txt
a
b
c
$ cat << EOF
> Test1
> Test2
> EOF
Test1
Test2
```

Thử nghiệm chuyển hướng đầu vào chuẩn (stdin) với < và <<

- < (**Chuyển hướng đầu vào**): Chuyển hướng tệp unsorted_list.txt làm đầu vào (stdin) cho lệnh sort xử lý thay vì chờ người dùng nhập từ bàn phím.
- << (**Here Document**): Cho phép chỉ định một chuỗi văn bản nhiều dòng làm đầu vào cho lệnh cat cho đến khi hệ thống gặp phải chuỗi phân cách (ở đây là EOF).

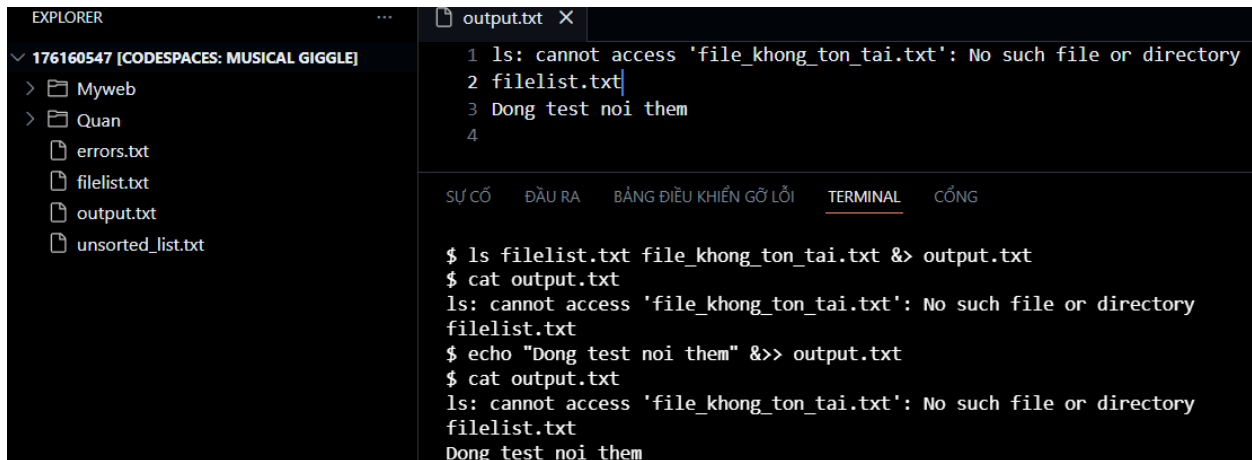


The screenshot shows a code editor interface with a file explorer on the left and a terminal on the right. The file explorer shows files 'errors.txt', 'filelist.txt', and 'unsorted_list.txt'. The terminal shows the following commands and output:

```
$ grep "something" file_khong_ton_tai.txt 2> errors.txt
$ cat errors.txt
grep: file_khong_ton_tai.txt: No such file or directory
$ grep "another" file_khong_ton_tai.txt 2>> errors.txt
$ cat errors.txt
grep: file_khong_ton_tai.txt: No such file or directory
grep: file_khong_ton_tai.txt: No such file or directory
$
```

Thử nghiệm chuyển hướng luồng lỗi (stderr) với 2> và 2>>

- **2> (Ghi đè luồng lỗi):** Số 2 đại diện cho Standard Error. Lệnh này bắt luồng lỗi của lệnh grep khi tìm file không tồn tại và chuyển hướng ghi đè đến tệp errors.txt thay vì in ra Terminal gây nhiễu thông tin.
- **2>> (Nối thêm luồng lỗi):** Nối thêm thông báo lỗi mới vào cuối tệp errors.txt (giữ nguyên lỗi cũ), tạo tệp nếu nó không tồn tại.



```
1 ls: cannot access 'file_khong_ton_tai.txt': No such file or directory
2 filelist.txt
3 Dong test noi them
4

SỰ CỐ   ĐẦU RA   BẢNG ĐIỀU KHIỂN GỠ LỖI   TERMINAL   CỔNG

$ ls filelist.txt file_khong_ton_tai.txt &> output.txt
$ cat output.txt
ls: cannot access 'file_khong_ton_tai.txt': No such file or directory
filelist.txt
$ echo "Dong test noi them" &>> output.txt
$ cat output.txt
ls: cannot access 'file_khong_ton_tai.txt': No such file or directory
filelist.txt
Dong test noi them
```

Thử nghiệm gộp chung đầu ra và lỗi với &> và &>>

- **&> (Ghi đè cả stdout và stderr):** Lệnh này gom chung cả kết quả thành công và thông báo lỗi lại, sau đó ghi đè đồng thời đến tệp output.txt.
- **&>> (Nối thêm cả stdout và stderr):** Tương tự lệnh trên nhưng nối thêm dữ liệu (cả luồng đúng và luồng lỗi) vào cuối tệp, tạo tệp nếu nó không tồn tại.

1.2 CLASSWORK 02:

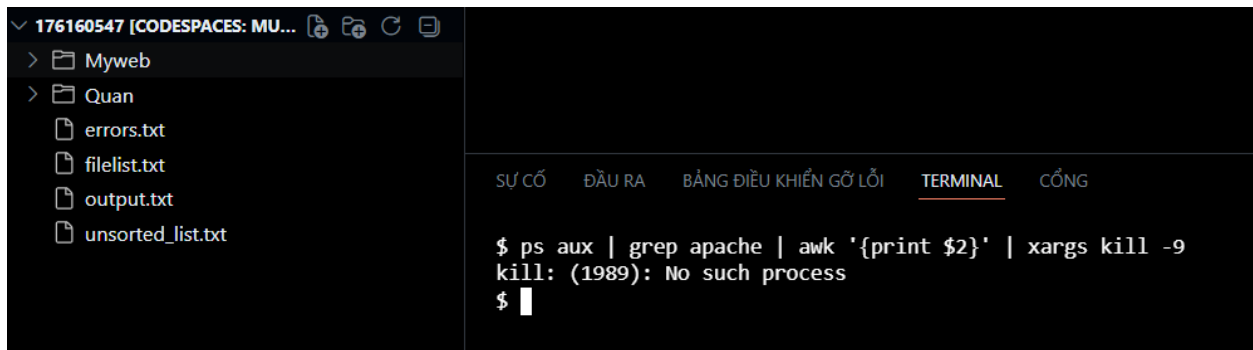
*Phương pháp thực hiện:

- **Phân tích cấu trúc:** Nhận diện câu lệnh sử dụng ký hiệu pipe (|) để tạo thành pipeline, giúp kết nối đầu ra của lệnh đứng trước thành đầu vào cho lệnh tiếp theo.

- **Tra cứu chức năng:** Dựa vào lý thuyết điều khiển tiến trình và xử lý văn bản của Linux để tìm hiểu chức năng của các lệnh con như grep, awk, xargs, kill.

- **Tổng hợp:** Lắp ghép chức năng của từng tiến trình nhỏ để diễn giải mục đích cuối cùng của toàn bộ chuỗi lệnh.

*Minh chứng và giải thích:



```
176160547 [CODESPACES: MU...]  
> Myweb  
> Quan  
  errors.txt  
  filelist.txt  
  output.txt  
  unsorted_list.txt  
  
SỰ CỐ  ĐẦU RA  BẢNG ĐIỀU KHIỂN GỖ LỖI  TERMINAL  CỔNG  
  
$ ps aux | grep apache | awk '{print $2}' | xargs kill -9  
kill: (1989): No such process  
$
```

Thực hiện câu lệnh `ps aux | grep apache | awk '{print $2}' | xargs kill -9`

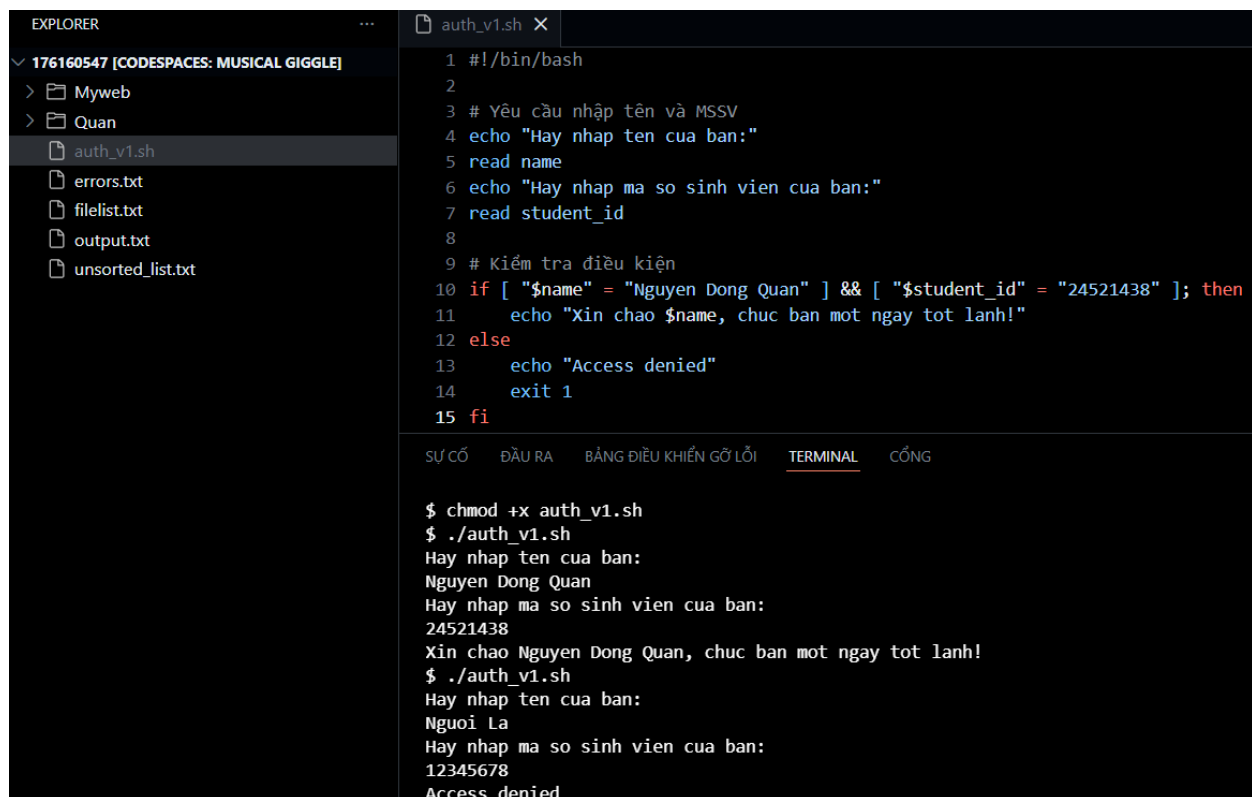
- **ps aux:** Lệnh ps được dùng để kiểm tra các tiến trình đang chạy trên hệ thống. Tổ hợp cờ aux giúp liệt kê danh sách toàn bộ các tiến trình đang hoạt động của tất cả người dùng một cách chi tiết (bao gồm cả thông tin ở cột mã tiến trình - PID).
- **| (Ký hiệu pipe):** Là ký tự điều khiển quá trình, làm nhiệm vụ kết nối đầu ra (stdout) của lệnh phía trước thành đầu vào (stdin) cho lệnh đứng sau.
- **grep apache:** grep là công cụ tìm kiếm và lọc văn bản. Lệnh này nhận dữ liệu từ ps aux và tiến hành lọc ra chỉ những dòng có chứa từ khóa "apache" (những tiến trình liên quan đến máy chủ web Apache).
- **awk '{print \$2}':** awk là lệnh dùng để xử lý và chuyển đổi văn bản. Lệnh này đọc các dòng chứa tiến trình "apache" vừa được lọc và chỉ trích xuất giá trị nằm ở cột thứ 2. Trong đầu ra mặc định của lệnh ps, cột thứ 2 chính là Process ID (PID - mã định danh tiến trình).
- **xargs kill -9:** xargs là lệnh xây dựng và thực thi các dòng lệnh từ đầu vào tiêu chuẩn. Nó tiếp nhận danh sách các PID từ lệnh awk phía trước và tự động biến chúng thành các đối số truyền vào cho lệnh kill -9 để thực hiện việc bắt buộc chấm dứt các tiến trình đó.
- **Ý nghĩa:** Câu lệnh trên được sử dụng để tìm kiếm tất cả các tiến trình có tên chứa "apache" đang chạy trên hệ điều hành, lấy ra danh sách mã tiến trình (PID) tương ứng của chúng, sau đó tự động và ngay lập tức ép buộc dừng (kill) toàn bộ các tiến trình đó cùng một lúc.
- **Lưu ý:** Việc sử dụng cờ -9 (SIGKILL) dễ gây mất dữ liệu hoặc lỗi tệp tin vì tiến trình không có thời gian để đóng một cách an toàn. Ngoài ra, lệnh này thường yêu cầu quyền **sudo** để thực thi và đôi khi xuất hiện lỗi giả "No such process" (do chính lệnh grep đã kết thúc trước khi bị kill), nhưng điều này không ảnh hưởng đến kết quả cuối cùng là Apache đã bị dừng.

1.3 CLASSWORK 03:

*Phương pháp thực hiện:

- **Tạo tệp kịch bản:** Khởi tạo một tệp Shell script mới (ví dụ: auth_v1.sh) với dòng shebang `#!/bin/bash` ở đầu để hệ điều hành biết dùng Bash shell để thực thi.
- **Lấy dữ liệu đầu vào:** Sử dụng lệnh `echo` để in ra câu hỏi và lệnh `read` để chờ người dùng nhập tên và mã số sinh viên từ bàn phím, gán vào các biến tương ứng.
- **Thiết lập cấu trúc điều kiện:** Sử dụng cấu trúc `if-else` kết hợp với toán tử so sánh chuỗi `=` và toán tử logic `&&` (AND) để kiểm tra xem dữ liệu nhập vào có khớp hoàn toàn với thông tin của mình hay không.
- **Phân quyền và thực thi:** Cấp quyền thực thi cho file script bằng lệnh `chmod +x` và tiến hành chạy thử với cả 2 trường hợp (nhập đúng và nhập sai) để đối chiếu kết quả.

*Minh chứng và giải thích:



```
EXPLORER
176160547 [CODESPACES: MUSICAL GIGGLE]
  > Myweb
  > Quan
    auth_v1.sh
    errors.txt
    filelist.txt
    output.txt
    unsorted_list.txt

auth_v1.sh
1 #!/bin/bash
2
3 # Yêu cầu nhập tên và MSSV
4 echo "Hay nhập ten cua ban:"
5 read name
6 echo "Hay nhập ma so sinh vien cua ban:"
7 read student_id
8
9 # Kiểm tra điều kiện
10 if [ "$name" = "Nguyen Dong Quan" ] && [ "$student_id" = "24521438" ]; then
11     echo "Xin chao $name, chuc ban mot ngay tot lanh!"
12 else
13     echo "Access denied"
14     exit 1
15 fi

SỰ CỐ    ĐẦU RA    BẢNG ĐIỀU KHIỂN GỖ LỖI    TERMINAL    CỔNG

$ chmod +x auth_v1.sh
$ ./auth_v1.sh
Hay nhập ten cua ban:
Nguyen Dong Quan
Hay nhập ma so sinh vien cua ban:
24521438
Xin chao Nguyen Dong Quan, chuc ban mot ngay tot lanh!
$ ./auth_v1.sh
Hay nhập ten cua ban:
Nguoi La
Hay nhập ma so sinh vien cua ban:
12345678
Access denied
```

Nội dung file kịch bản auth_v1.sh

- **read name** và **read student_id**: Lệnh `read` dùng để bắt thông tin người dùng vừa nhập từ bàn phím (Standard Input) và lưu trữ giá trị đó vào các biến shell (như `name`, `student_id`) để dùng cho các bước xử lý sau.

- ["\$name" = "Nguyễn Dong Quan"]: Trong shell, cặp ngoặc vuông [] đại diện cho lệnh kiểm tra điều kiện (test). Ký hiệu = được dùng để so sánh hai chuỗi xem chúng có giống hệt nhau hay không. Chú ý phải có khoảng trắng ở cả hai bên dấu =.
- && (AND): Là toán tử logic dùng để gộp nhiều điều kiện lại với nhau. Toàn bộ biểu thức chỉ đúng (true) khi cả thông tin Tên và Mã số sinh viên đều nhập chính xác.
- if ... then ... else ... fi: Là cấu trúc điều kiện phân nhánh. Nếu tài khoản nhập đúng (khớp thông tin), script sẽ chạy lệnh echo trong khối then để in ra lời chào. Nếu nhập sai, nó sẽ nhảy xuống khối else in ra "Access denied".
- exit 1: Lệnh kết thúc chương trình ngay lập tức. Số 1 biểu thị cho hệ điều hành biết rằng chương trình đã thoát ra do gặp lỗi (ở đây là lỗi xác thực thất bại), thay vì thoát bình thường (thường là exit 0).

1.4 CLASSWORK 04:

*Phương pháp thực hiện:

- **Tạo tệp kịch bản:** Khởi tạo file script mới (ví dụ: grade_converter.sh) với dòng lệnh shebang #!/bin/bash ở đầu để hệ thống nhận diện loại shell.
- **Tiếp nhận biến tham số:** Thay vì dùng lệnh read, ta sử dụng biến tham số đặc biệt \$1 để lấy giá trị điểm số được truyền vào ngay từ dòng lệnh khi khởi chạy script.
- **Thiết lập luồng rẽ nhánh:** Sử dụng cấu trúc rẽ nhánh if-elif-else kết hợp với các toán tử so sánh toán học (-ge - lớn hơn hoặc bằng, -lt - nhỏ hơn, -le - nhỏ hơn hoặc bằng).
- **Kết hợp đa điều kiện:** Dùng các toán tử logic AND để giới hạn điểm số rơi vào đúng các khoảng quy định trong Bảng 1 của đề bài.
- **Phân quyền và kiểm tra thực thi:** Dùng lệnh chmod +x để cấp quyền thực thi và chạy thử với các tham số mẫu.

*Minh chứng và giải thích:

The screenshot shows a code editor with two tabs: `auth_v1.sh` and `grade_converter.sh`. The left sidebar shows a file explorer with a directory structure including `Myweb`, `Quan`, and several files like `auth_v1.sh`, `errors.txt`, `filelist.txt`, `grade_converter.sh`, `output.txt`, and `unsorted_list.txt`. The main editor displays the content of `grade_converter.sh`, which is a shell script for converting scores to letter grades. Below the editor, a terminal window shows the execution of the script with various scores and the resulting letter grades.

```

1  #!/bin/bash
2
3  score=$1
4
5  # Kiểm tra xem người dùng đã truyền tham số hay chưa
6  if [ -z "$score" ]; then
7      echo "Vui long nhap diem so vao dong lenh!"
8      exit 1
9  fi
10
11 # So sánh để quy đổi điểm số
12 if [ $score -ge 900 -a $score -le 1000 ]; then
13     echo "A+"
14 elif [ $score -ge 800 ] && [ $score -lt 900 ]; then
15     echo "A"
16 elif [ $score -ge 700 ] && [ $score -lt 800 ]; then
17     echo "B+"
18 elif [ $score -ge 600 ] && [ $score -lt 700 ]; then
19     echo "B"
20 elif [ $score -ge 500 ] && [ $score -lt 600 ]; then
21     echo "C"
22 elif [ $score -lt 500 ]; then
23     echo "D/F"
24 else
25     echo "Diem khong hop le"
26 fi
    
```

Terminal output:

```

$ chmod +x grade_converter.sh
$ ./grade_converter.sh 700
B+
$ ./grade_converter.sh 850
A
$ ./grade_converter.sh 450
D/F
    
```

Nội dung file kịch bản `grade_converter.sh`

- **score=\$1**: Biến `$1` đại diện cho biến tham số thứ nhất truyền vào kịch bản từ dòng lệnh. Ví dụ khi gõ `./grade_converter.sh 700` thì `$1` sẽ mang giá trị là 700.
- **[-z "\$score"]**: Điều kiện kiểm tra chuỗi rỗng. Cú pháp này trả về true nếu độ dài của biến `$score` bằng 0 (tức chạy file script nhưng quên nhập điểm số kèm theo).
- **-ge, -le, -lt**: Lần lượt là các biểu thức so sánh toán học mang ý nghĩa: lớn hơn hoặc bằng (greater or equal), nhỏ hơn hoặc bằng (less or equal) và nhỏ hơn (less than).
- **-a và &&**: Cả hai đều là toán tử AND dùng để thiết lập đa điều kiện. Kịch bản sử dụng linh hoạt cả hai cách viết: dùng `-a` chung trong một cặp ngoặc vuông `[ $score -ge 900 -a`

`$score -le 1000 ]` hoặc tách ra dùng `&&` giữa hai ngoặc vuông `[ $score -ge 800 ] && [ $score -lt 900 ]` để lọc chính xác khoảng điểm quy đổi.

1.5 CLASSWORK 05:

***Phương pháp thực hiện:**

- **Phân tích yêu cầu:** Kế thừa bài toán từ Classwork 03, yêu cầu lần này chia làm 2 trường hợp nâng cao: (1) lặp vô tận bằng `while` cho đến khi nhập đúng và (2) giới hạn 5 lần nhập bằng `for`.

- **Khởi tạo kịch bản (Script):** Khởi tạo 2 file script riêng biệt `auth_while.sh` và `auth_for.sh` với shebang `#!/bin/bash`.

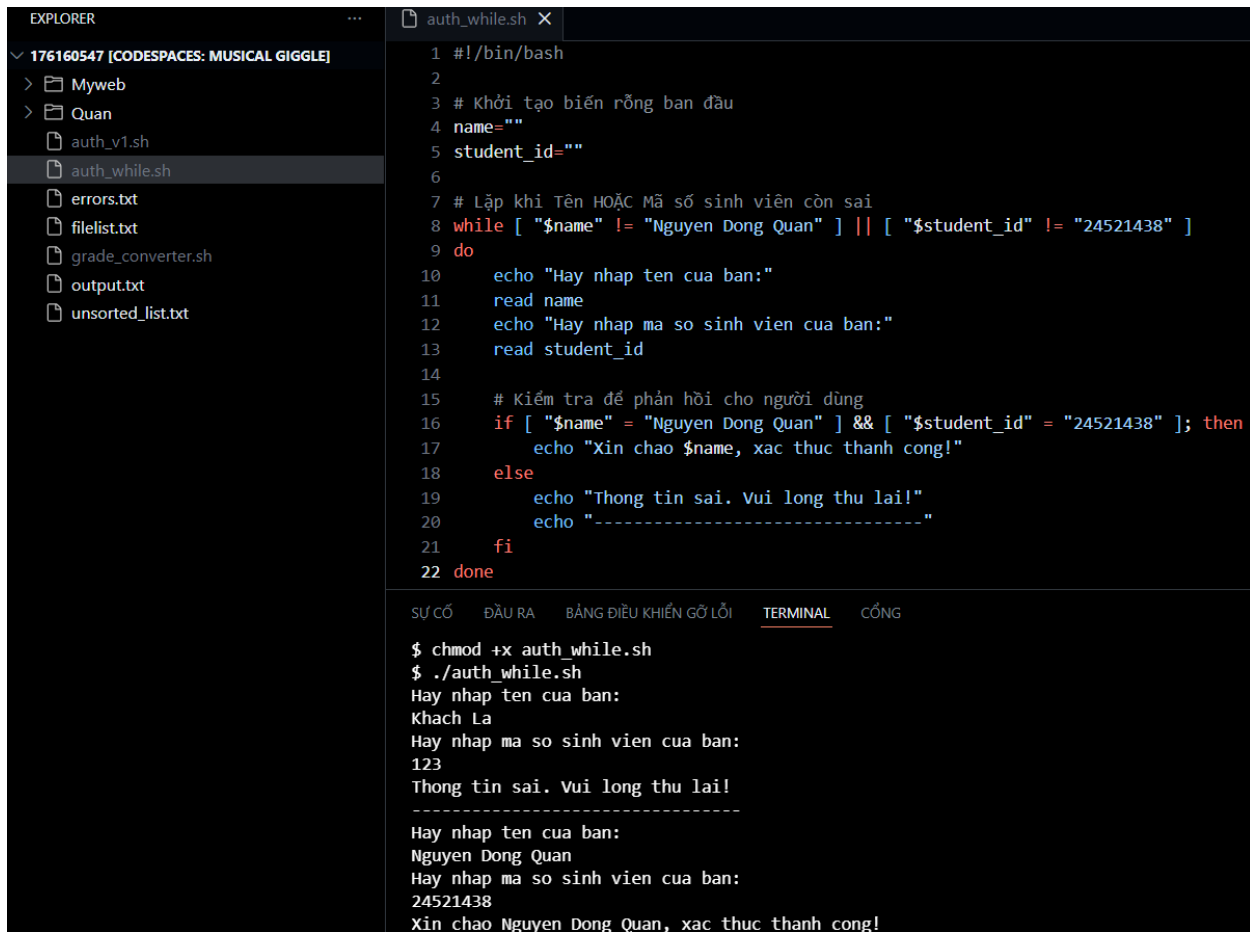
- Áp dụng vòng lặp:

- Trường hợp 1: Sử dụng cấu trúc `while [ condition ]` kết hợp toán tử logic để lặp liên tục nếu thông tin sai.
- Trường hợp 2: Sử dụng cấu trúc vòng lặp `for` kiểu C (`for ((i=1; i<=5; i++))`) để đếm số lần nhập tối đa.

- **Thực thi và kiểm tra:** Cấp quyền bằng `chmod +x` cho cả 2 tệp, tiến hành chạy thử nghiệm nhập sai nhiều lần để quan sát luồng điều khiển của từng script.

***Minh chứng và giải thích:**

A. Kịch bản 1 - Sử dụng vòng lặp `while` (Nhập đến khi đúng)



The screenshot displays a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a directory structure with files like `auth_v1.sh`, `auth_while.sh`, `errors.txt`, `filelist.txt`, `grade_converter.sh`, `output.txt`, and `unsorted_list.txt`. The code editor shows the content of `auth_while.sh`, which is a shell script that prompts the user for their name and student ID, and checks if they match the expected values. The terminal output shows the script being executed, with the user entering "Khach La" and "123", which results in an error message. The user then enters "Nguyen Dong Quan" and "24521438", which results in a success message.

```
1 #!/bin/bash
2
3 # Khởi tạo biến rỗng ban đầu
4 name=""
5 student_id=""
6
7 # Lặp khi Tên HOẶC Mã số sinh viên còn sai
8 while [ "$name" != "Nguyen Dong Quan" ] || [ "$student_id" != "24521438" ]
9 do
10     echo "Hay nhap ten cua ban:"
11     read name
12     echo "Hay nhap ma so sinh vien cua ban:"
13     read student_id
14
15     # Kiểm tra để phản hồi cho người dùng
16     if [ "$name" = "Nguyen Dong Quan" ] && [ "$student_id" = "24521438" ]; then
17         echo "Xin chào $name, xác thực thành công!"
18     else
19         echo "Thông tin sai. Vui lòng thu lại!"
20         echo "-----"
21     fi
22 done
```

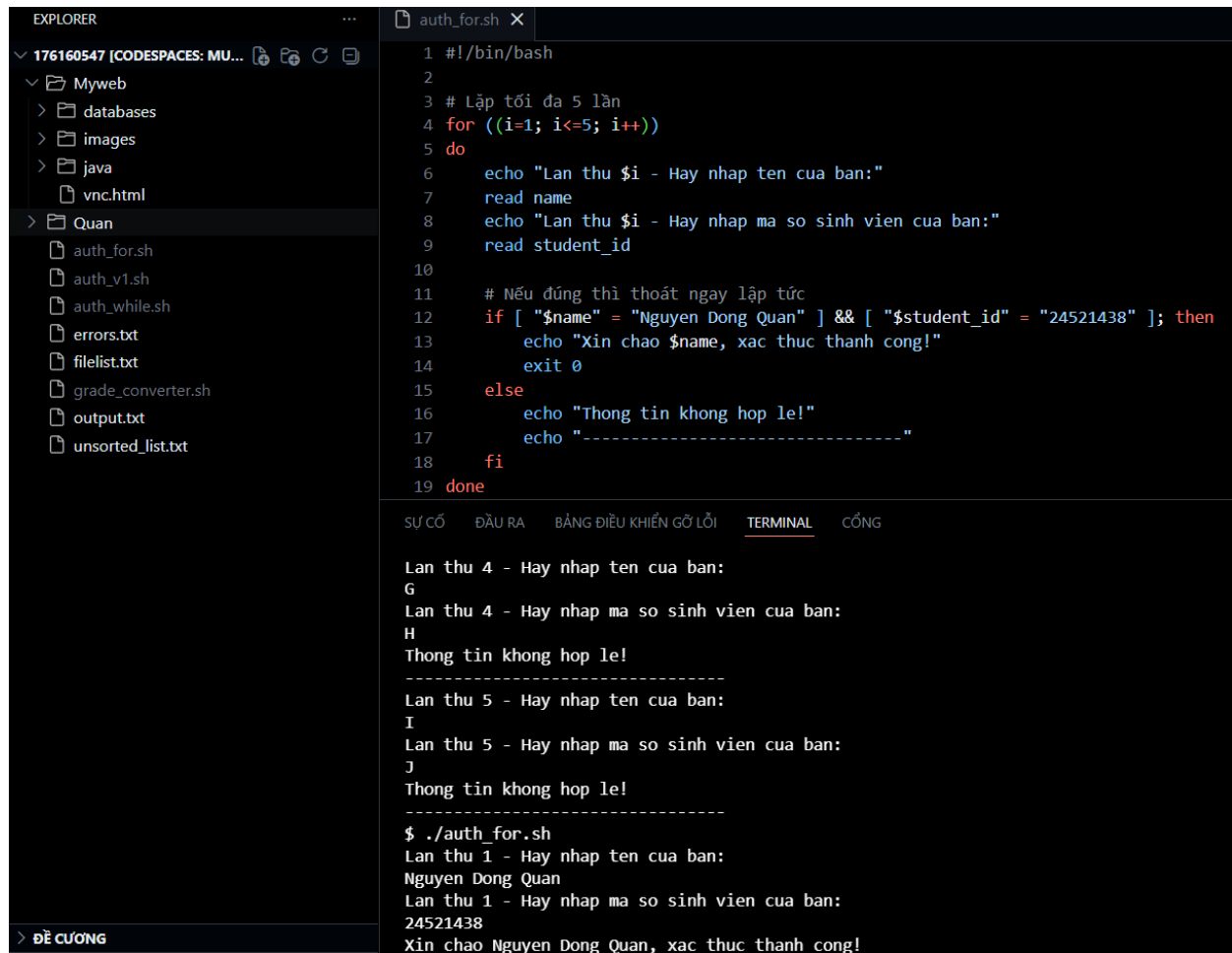
SỰ CỐ ĐẦU RA BẢNG ĐIỀU KHIỂN GỖ LỖI TERMINAL CỐNG

```
$ chmod +x auth_while.sh
$ ./auth_while.sh
Hay nhap ten cua ban:
Khach La
Hay nhap ma so sinh vien cua ban:
123
Thông tin sai. Vui lòng thu lại!
-----
Hay nhap ten cua ban:
Nguyen Dong Quan
Hay nhap ma so sinh vien cua ban:
24521438
Xin chào Nguyen Dong Quan, xác thực thành công!
```

Nội dung file kịch bản `auth_while.sh` và quá trình thực thi

Giải thích: Cấu trúc `while` sẽ đánh giá điều kiện sau chữ "`while`". Ở đây ta dùng OR (`||`), nghĩa là vòng lặp sẽ tiếp tục chạy chừng nào điều kiện tên (`name`) khác "Nguyen Dong Quan" hoặc MSSV (`student_id`) khác "24521438" vẫn còn đang đúng (nghĩa là nhập sai 1 trong 2 hoặc sai cả 2). Vòng lặp chỉ dừng khi cả 2 điều kiện đều sai (nghĩa là đã nhập đúng cả Tên và MSSV).

B. Kịch bản 2 - Sử dụng vòng lặp for (Cho phép thử 5 lần)



The screenshot shows a code editor with a file explorer on the left and a terminal on the right. The file explorer shows a directory structure with a folder named 'Quan' containing the file 'auth_for.sh'. The terminal displays the execution of 'auth_for.sh', which prompts the user for a name and student ID. It shows three failed attempts where the input does not match the expected values ('Nguyen Dong Quan' and '24521438'). After the third failure, the script prompts for a name and student ID again, and the user enters the correct values, resulting in a success message: 'Xin chào Nguyen Dong Quan, xác thực thành công!'.

```
1 #!/bin/bash
2
3 # Lặp tối đa 5 lần
4 for ((i=1; i<=5; i++))
5 do
6     echo "Lan thu $i - Hay nhap ten cua ban:"
7     read name
8     echo "Lan thu $i - Hay nhap ma so sinh vien cua ban:"
9     read student_id
10
11 # Nếu đúng thì thoát ngay lập tức
12 if [ "$name" = "Nguyen Dong Quan" ] && [ "$student_id" = "24521438" ]; then
13     echo "Xin chào $name, xác thực thành công!"
14     exit 0
15 else
16     echo "Thông tin không hợp lệ!"
17     echo "-----"
18 fi
19 done
```

SỰ CỐ ĐẦU RA BẢNG ĐIỀU KHIỂN GỖ LỖI TERMINAL CỔNG

```
Lan thu 4 - Hay nhap ten cua ban:
G
Lan thu 4 - Hay nhap ma so sinh vien cua ban:
H
Thông tin không hợp lệ!
-----
Lan thu 5 - Hay nhap ten cua ban:
I
Lan thu 5 - Hay nhap ma so sinh vien cua ban:
J
Thông tin không hợp lệ!
-----
$ ./auth_for.sh
Lan thu 1 - Hay nhap ten cua ban:
Nguyen Dong Quan
Lan thu 1 - Hay nhap ma so sinh vien cua ban:
24521438
Xin chào Nguyen Dong Quan, xác thực thành công!
```

Nội dung file kịch bản `auth_for.sh`

```

EXPLORER
176160547 [CODESPACES: MU...
  Myweb
    databases
    images
    java
      vnc.html
    Quan
      auth_for.sh
      auth_v1.sh
      auth_while.sh
      errors.txt
      filelist.txt
      grade_converter.sh
      output.txt
      unsorted_list.txt

SỰ CỐ  ĐẦU RA  BẢNG ĐIỀU KHIỂN GỖ LỖI  TERMINAL  CỔNG

$ chmod +x auth_for.sh
$ ./auth_for.sh
Lan thu 1 - Hay nhap ten cua ban:
A
Lan thu 1 - Hay nhap ma so sinh vien cua ban:
B
Thong tin khong hop le!
-----
Lan thu 2 - Hay nhap ten cua ban:
C
Lan thu 2 - Hay nhap ma so sinh vien cua ban:
D
Thong tin khong hop le!
-----
Lan thu 3 - Hay nhap ten cua ban:
E
Lan thu 3 - Hay nhap ma so sinh vien cua ban:
F
Thong tin khong hop le!
-----
Lan thu 4 - Hay nhap ten cua ban:
G
Lan thu 4 - Hay nhap ma so sinh vien cua ban:
H
Thong tin khong hop le!
-----
Lan thu 5 - Hay nhap ten cua ban:
I
Lan thu 5 - Hay nhap ma so sinh vien cua ban:
J
Thong tin khong hop le!
-----
$ ./auth_for.sh
Lan thu 1 - Hay nhap ten cua ban:
Nguyen Dong Quan
Lan thu 1 - Hay nhap ma so sinh vien cua ban:
24521438
Xin chao Nguyen Dong Quan, xac thuc thanh cong!
  
```

Quá trình thực thi file kịch bản auth_for.sh

Giải thích: Vòng lặp for kiểu C (for ((i=1; i<=5; i++))) trên Bash shell thiết lập số lần lặp định trước. Ở mỗi vòng lặp, ta yêu cầu người dùng nhập liệu và kiểm tra bằng if. Nếu nhập đúng, lệnh exit 0 sẽ kết thúc hoàn toàn chương trình (hiển nhiên thoát khỏi vòng lặp). Nếu người dùng nhập sai liên tục, biến i sẽ tăng dần lên 6, phá vỡ điều kiện lặp i<=5. Lúc này, script rơi xuống dòng lệnh bên dưới vòng for, in ra thông báo "Access denied" và dùng exit 1 để kết thúc do lỗi xác thực.

2. HOMEWORK

2.1 HOMEWORK 01:

*Phương pháp thực hiện:

- **Phân tích yêu cầu:** Đề bài yêu cầu liệt kê nội dung thư mục /etc kèm chi tiết chế độ trên file (quyền truy cập), sau đó chuyển hướng kết quả này vào một tệp tin mang tên report_<studentID>.txt nằm tại thư mục chính (home directory). Với mã số sinh viên là 24521438, tệp đích sẽ là report_24521438.txt.

- **Thực thi lệnh:** Sử dụng lệnh ls với tham số -l để hiển thị định dạng dài (có chứa thông tin phân quyền) đối với thư mục /etc. Sau đó kết hợp toán tử chuyển hướng > để đẩy đầu ra vào tệp ~/report_24521438.txt.

- **Kiểm tra kết quả:** Sử dụng lệnh ls để xác nhận tệp đã được tạo trong thư mục chính và lệnh head để xem thử vài dòng đầu tiên của tệp, đảm bảo dữ liệu đã được ghi thành công.

*Minh chứng và giải thích:

```
$ ls -l /etc
total 736
-rw-r--r-- 1 root    root    3444 Jul  6  2023 adduser.conf
drwxr-xr-x 1 root    root    4096 Mar 12 20:06 alternatives/
drwxr-xr-x 1 root    root    4096 Mar 12 20:05 apparmor.d/
drwxr-xr-x 1 root    root    4096 Jan 13 09:04 apt/
-rw-r--r-- 1 root    root    2365 Jan 21 13:30 bash.bashrc
-rw-r--r-- 1 root    root      45 Jan 25  2020 bash_completion
drwxr-xr-x 1 root    root    4096 Mar 12 20:06 bash_completion.d/
-rw-r--r-- 1 root    root    367 Aug  2  2022 bindresvport.blacklist
drwxr-xr-x 3 root    root    4096 Jan 21 01:59 ca-certificates/
-rw-r--r-- 1 root    root   6288 Jan 21 01:59 ca-certificates.conf
drwxr-xr-x 2 root    root    4096 Jan 13 09:11 cloud/
drwxr-xr-x 1 root    root    4096 Mar 12 20:05 cron.d/
```

Liệt kê nội dung của thư mục /etc bằng ls -l /etc

SỰ CỐ ĐẦU RA BẢNG ĐIỀU KHIỂN GỖ LỖI TERMINAL CỔNG

```
$ ls -l /etc > ~/report_24521438.txt
$ ls -l ~/report_24521438.txt
-rw----- 1 ubuntu ubuntu 9078 Mar 18 00:48 /home/ubuntu/report_24521438.txt
$
```

Thực thi chuyển hướng đầu ra theo yêu cầu đề bài

- **ls -l /etc:** Lệnh ls dùng để liệt kê danh sách tệp tin. Tham số -l (long-listing format) chỉ định in ra chi tiết các thông tin như: chế độ trên file (permissions), số lượng liên kết, người sở hữu (owner), nhóm sở hữu (group), kích thước và ngày chỉnh sửa. /etc là thư mục chứa các tệp cấu hình hệ thống cần quét.
- **> (Ký tự chuyển hướng đầu ra):** Ký hiệu này sẽ bắt luồng đầu ra tiêu chuẩn (stdout) của lệnh ls -l /etc thay vì in lướt trên màn hình Terminal, nó sẽ đẩy toàn bộ văn bản đó và ghi đè vào tệp tin đích. Nếu tệp chưa tồn tại, hệ thống sẽ tự động tạo mới.
- **~/report_24521438.txt:** Dấu ngã ~ là ký hiệu đại diện (shortcut) cho thư mục chính (Home directory) của người dùng hiện hành (tương đương với đường dẫn sinh ra từ biến môi trường \$HOME). report_24521438.txt là tên tệp được đặt theo đúng cú pháp đề bài yêu cầu report_<studentID>.txt.

2.2 HOMEWORK 02:

*Phương pháp thực hiện:

- **Phân tích yêu cầu:** Đề bài yêu cầu sắp xếp nội dung của tệp report_24521438.txt (đã tạo ở Homework 01) và nối kết quả vào cuối một tệp mới có tên là report-sorted.txt.
- **Thực thi lệnh:** Sử dụng lệnh sort để sắp xếp các dòng văn bản. Kết hợp với toán tử chuyển hướng >> để nối thêm đầu ra (stdout) vào tệp đích report-sorted.txt mà không làm mất dữ liệu nếu tệp đã tồn tại.
- **Kiểm tra kết quả:** Sử dụng lệnh head hoặc cat để xem thử tệp report-sorted.txt nhằm xác nhận nội dung đã được sắp xếp theo bảng chữ cái.

*Minh chứng và giải thích:

```
$ sort ~/report_24521438.txt >> ~/report-sorted.txt
$ head -n 5 ~/report-sorted.txt
drwxr-xr-x 1 postgres postgres 4096 Jan 21 13:30 postgresql/
drwxr-xr-x 1 root    root      4096 Dec  6 2023 systemd/
drwxr-xr-x 1 root    root      4096 Jan 13 09:04 apt/
drwxr-xr-x 1 root    root      4096 Jan 21 01:59 perl/
drwxr-xr-x 1 root    root      4096 Jan 21 02:00 logcheck/
$
```

Thực thi dòng lệnh trên Terminal và kiểm tra tệp tin đã được tạo chứa dữ liệu đã sắp xếp

- **sort ~/report_24521438.txt:** Lệnh sort được dùng để sắp xếp các dòng của tệp văn bản theo thứ tự từ điển (mặc định). Tệp đầu vào ở đây là report_24521438.txt.
- **>> ~/report-sorted.txt:** Toán tử >> dùng để bắt đầu ra chuẩn của lệnh sort và chuyển hướng nối thêm vào phần cuối của tệp report-sorted.txt. Vì trước đó tệp này chưa tồn tại, hệ thống sẽ tự động tạo tệp mới và đưa dữ liệu vào.
- **head -n 5 ~/report-sorted.txt:** Lệnh xem thử 5 dòng đầu tiên của tệp để xác nhận nội dung đã được sắp xếp.

2.3 HOMEWORK 03:

*Phương pháp thực hiện:

- **Phân tích yêu cầu:** Đề bài yêu cầu đếm số từ trong tệp report_24521438.txt, sau đó ghi kết quả nối tiếp vào tệp field2.txt nằm ở thư mục chính (home). Yêu cầu bắt buộc là phải sử dụng cả chuyển hướng đầu vào và đầu ra trong cùng một câu lệnh.

- **Thực thi lệnh:** Sử dụng lệnh wc (Word Count) kết hợp tham số -w để đếm từ. Dùng ký tự < để chuyển hướng đầu vào (đưa vào lệnh wc) và ký tự >> để nối đầu ra (xuất kết quả ra file field2.txt).

- **Kiểm tra kết quả:** Dùng lệnh cat ~/field2.txt để xem kết quả đếm số từ.

*Minh chứng và giải thích:

```
$ wc -w < ~/report_24521438.txt >> ~/field2.txt
$ cat ~/field2.txt
1243
$
```

Thực thi dòng lệnh trên Terminal theo yêu cầu đề bài

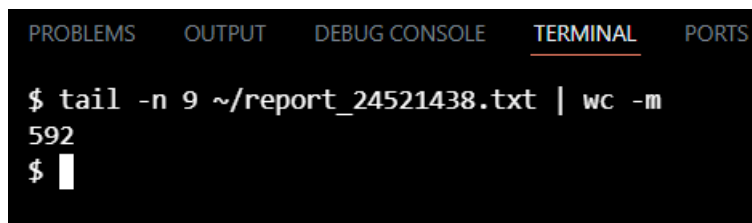
- **wc -w:** Lệnh wc (Word Count) chuyên dùng để đếm số dòng, số từ hoặc số ký tự. Khi truyền thêm tham số -w (words), lệnh sẽ chỉ hiển thị riêng số lượng từ.
- **< ~/report_24521438.txt:** Đây là thao tác chuyển hướng đầu vào (stdin). Thay vì truyền tên file như một đối số (điều này sẽ khiến lệnh in ra cả kết quả đếm kèm tên file), ta chuyển hướng luồng dữ liệu của tệp trực tiếp vào lệnh wc. Nhờ đó kết quả trả về chỉ là một con số thuần túy (ví dụ: 12543).
- **>> ~/field2.txt:** Là thao tác chuyển hướng đầu ra (stdout). Kết quả sẽ không in ra màn hình Terminal mà được hệ thống nối thêm vào tệp tin field2.txt ở thư mục chính.

2.4 HOMEWORK 04:

*Phương pháp thực hiện:

- **Phân tích yêu cầu:** Đề bài yêu cầu lấy ra 9 dòng cuối cùng của tệp report_24521438.txt và đếm tổng số lượng ký tự có trong 9 dòng đó.
- **Thực thi lệnh:** Sử dụng lệnh tail với tham số -n 9 để trích xuất 9 dòng cuối cùng,. Sau đó, sử dụng ký tự điều khiển đường ống (Pipeline |) để kết nối đầu ra của tail thành đầu vào cho lệnh wc (Word Count). Ta dùng lệnh wc kèm theo tham số -m (hoặc -c) để đếm số lượng ký tự.
- **Kiểm tra kết quả:** Nhấn Enter để xem con số thống kê được in ra màn hình.

*Minh chứng và giải thích:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

$ tail -n 9 ~/report_24521438.txt | wc -m
592
$
```

Thực thi dòng lệnh trên Terminal:

- **tail -n 9 ~/report_24521438.txt:** Lệnh tail bắt đầu đọc dữ liệu từ dòng cuối cùng của tệp tin ngược lên trên. Tham số -n 9 chỉ định lấy chính xác 9 dòng cuối.
- **| (Pipeline):** Ký tự Pipe giúp chuyển hướng đầu ra (stdout) của lệnh phía trước (tail) thẳng làm đầu vào (stdin) cho lệnh phía sau (wc) thay vì in ra màn hình.
- **wc -m:** Lệnh wc dùng để đếm dòng, từ và ký tự. Tham số -m (hoặc -c) yêu cầu chỉ đếm và hiển thị duy nhất số lượng ký tự (characters) của luồng dữ liệu được truyền vào.

2.5 HOMEWORK 05:

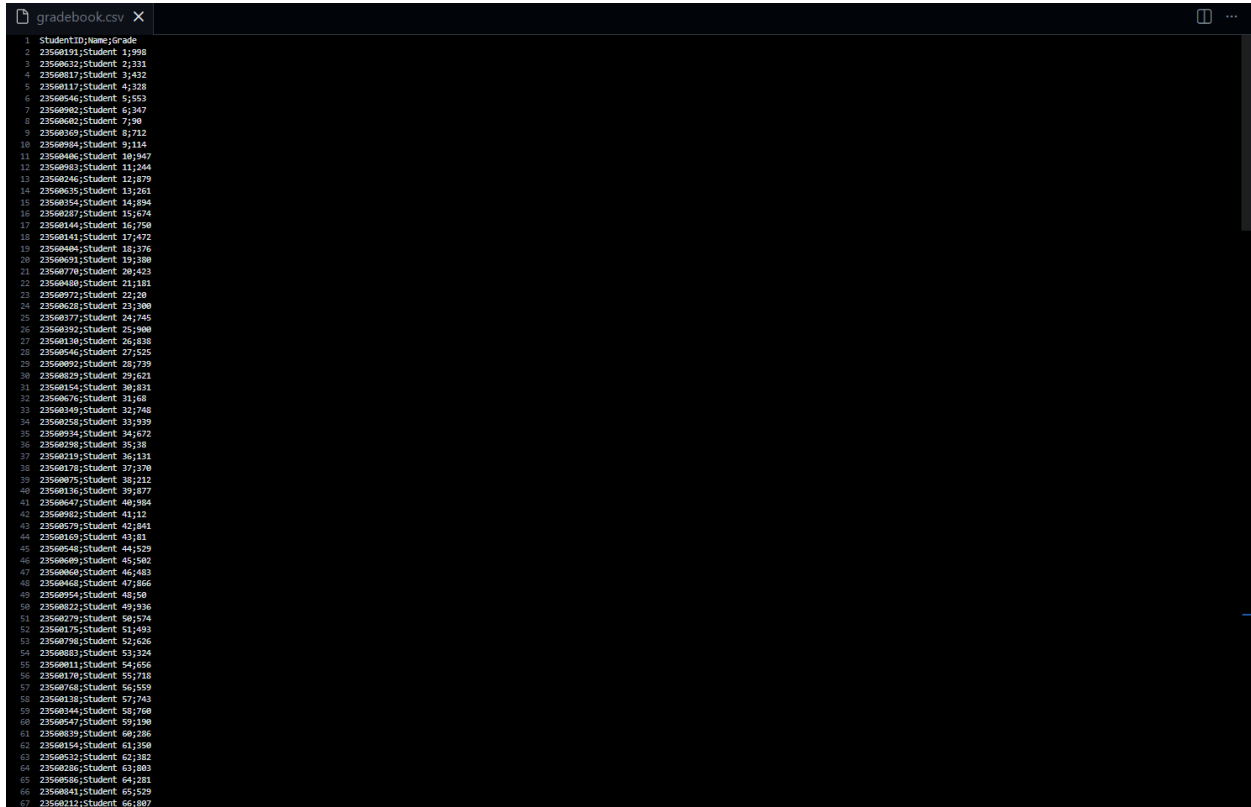
*Phương pháp thực hiện:

- **Phân tích dữ liệu:** Tệp gradebook.csv được cung cấp chứa danh sách điểm của lớp với định dạng StudentID;Name;Grade. Ký tự phân cách ở đây là dấu chấm phẩy (;).
- **Khởi tạo kịch bản:** Viết script find_student.sh nhận ID sinh viên từ dòng lệnh thông qua biến tham số \$1.
- **Sử dụng vòng lặp và IFS:** Dùng vòng lặp while kết hợp với biến môi trường IFS=";" (Internal Field Separator) để chia đầu vào thành 3 biến id, name, score khi đọc từng dòng của file.

- **Xử lý điều kiện:** So sánh biến \$id đọc được với tham số \$1 tìm kiếm. Nếu khớp, sử dụng cấu trúc if-elif-else và các toán tử so sánh toán học (-ge, -lt, -le) để quy đổi điểm gốc sang điểm chữ dựa trên **Bảng 1** (A+: 900-1000, A: 800-899, B+: 700-799, B: 600-699, C: 500-599, D/F: <500). Cuối cùng in kết quả ra màn hình.

- **Thực thi:** Cấp quyền thực thi bằng lệnh chmod +x và khởi chạy file kịch bản find_student.sh.

***Minh chứng và giải thích:**



```
1 studentID;Grade
2 235600191;student 1;990
3 235600132;student 2;131
4 235600117;student 3;432
5 235600117;student 4;128
6 235600546;student 5;553
7 235600902;student 6;347
8 235600602;student 7;90
9 235600369;student 8;712
10 235600904;student 9;114
11 235600406;student 10;947
12 235600903;student 11;244
13 235600246;student 12;679
14 235600235;student 13;261
15 235600354;student 14;894
16 235600287;student 15;674
17 235600144;student 16;794
18 235600141;student 17;472
19 235600404;student 18;376
20 235600091;student 19;300
21 235600770;student 20;421
22 235600480;student 21;181
23 235600972;student 22;20
24 235600828;student 23;900
25 235600377;student 24;745
26 235600392;student 25;900
27 23560130;student 26;130
28 235600445;student 27;525
29 235600092;student 28;739
30 23560029;student 29;621
31 23560115;student 30;131
32 235600676;student 31;48
33 235600349;student 32;748
34 235600258;student 33;939
35 235600934;student 34;672
36 235600200;student 35;18
37 235600219;student 36;131
38 235600178;student 37;370
39 235600079;student 38;212
40 235600136;student 39;877
41 235600647;student 40;904
42 235600082;student 41;12
43 23560079;student 42;041
44 23560169;student 43;81
45 235600548;student 44;529
46 235600609;student 45;902
47 235600660;student 46;483
48 235600468;student 47;866
49 235600954;student 48;90
50 235600222;student 49;196
51 235600279;student 50;574
52 235600175;student 51;493
53 235600708;student 52;626
54 235600083;student 53;324
55 235600011;student 54;656
56 235600170;student 55;718
57 235600706;student 56;559
58 235600138;student 57;743
59 235600344;student 58;760
60 235600547;student 59;190
61 23560039;student 60;286
62 235600154;student 61;350
63 23560032;student 62;382
64 235600236;student 63;803
65 235600586;student 64;281
66 235600431;student 65;529
67 235600211;student 66;807
```

Hình ảnh file gradebook.csv được xem dưới dạng plain text

```

gradebook.csv  find_student.sh X
1  #!/bin/bash
2
3  # Lấy ID từ tham số dòng lệnh
4  student_id=$1
5
6  # Kiểm tra nếu người dùng không truyền tham số
7  if [ -z "$student_id" ]; then
8      echo "Lỗi: Vui lòng truyền ID sinh viên tu dong lenh!"
9      exit 1
10 fi
11
12 # Đọc file gradebook.csv bằng vòng lặp while và đặt IFS=";"
13 while IFS=";" read -r id name score
14 do
15     # Bỏ qua dòng tiêu đề "StudentID;Name;Grade"
16     if [ "$id" = "StudentID" ]; then
17         continue
18     fi
19
20     # Nếu ID đọc được khớp với ID cần tìm kiếm
21     if [ "$id" = "$student_id" ]; then
22
23         # Quy đổi điểm chữ theo Bảng 1
24         if [ $score -ge 900 -a $score -le 1000 ]; then
25             grade="A+"
26         elif [ $score -ge 800 ] && [ $score -lt 900 ]; then
27             grade="A"
28         elif [ $score -ge 700 ] && [ $score -lt 800 ]; then
29             grade="B+"
30         elif [ $score -ge 600 ] && [ $score -lt 700 ]; then
31             grade="B"
32         elif [ $score -ge 500 ] && [ $score -lt 600 ]; then
33             grade="C"
34         elif [ $score -lt 500 ]; then
35             grade="D/F"
36         else
37             grade="Khong hop le"
38         fi
39
40         echo "Sinh vien: $name | Diem goc: $score | Diem chu: $grade"
41         exit 0
42     fi
43 done < "gradebook.csv"
44
45 # Nếu chạy hết vòng lặp mà không gặp lệnh exit 0, nghĩa là không tìm thấy
46 echo "Khong tim thay sinh vien co ID $student_id"

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

$ chmod +x find_student.sh
$ ./find_student.sh 23560191
Sinh vien: Student 1 | Diem goc: 998 | Diem chu: A+
$ ./find_student.sh 23560246
Sinh vien: Student 12 | Diem goc: 879 | Diem chu: A
$ ./find_student.sh 23560602
Sinh vien: Student 7 | Diem goc: 90 | Diem chu: D/F
$ ./find_student.sh 99999999
Khong tim thay sinh vien co ID 99999999

```

Thực thi chạy thử nghiệm file kịch bản find_student.sh trên Terminal

- **IFS=";":** Đây là khai báo quan trọng nhất trong bài này. Theo tệp dữ liệu được cung cấp, các trường được cách nhau bởi dấu chấm phẩy. Khi đặt IFS=";", lệnh read sẽ biết cách lấy phần trước dấu ; thứ nhất đưa vào biến id, phần trước dấu ; thứ 2 đưa vào biến name, và phần còn lại đưa vào biến score.
- **continue:** Lệnh này được dùng để yêu cầu vòng lặp while lập tức bỏ qua dòng hiện tại và nhảy sang chu kỳ lặp kế tiếp. Ở đây tôi thêm một bước lọc nhỏ if ["\$id" = "StudentID"] nhằm bỏ qua việc xử lý dòng tiêu đề ngoài cùng của file gradebook.csv.
- **Chuỗi lệnh rẽ nhánh if-elif-else và đa điều kiện ([\$score -ge ...] && [\$score -lt ...]):** Đây là cấu trúc dùng để xét duyệt toàn bộ các khoảng điểm quy đổi. Lệnh sử dụng các toán tử so sánh toán học (như -ge: lớn hơn hoặc bằng, -lt: nhỏ hơn) kết hợp với toán tử AND (&& hoặc -a) để ép giá trị \$score vào đúng từng khoảng. Các dòng lệnh elif hoạt động tương tự nhau, liên tục đối chiếu điểm số từ cao xuống thấp theo đúng chuẩn của Bảng 1 để gán ra điểm chữ tương ứng (từ A+ xuống D/F) cho biến grade.

2.6 HOMEWORK 06:

*Phương pháp thực hiện:

- **Khởi tạo kịch bản:** Viết script check_age.sh, dùng read để yêu cầu nhập tuổi.
- **Xác thực và phân loại:** Sử dụng toán tử so sánh toán học như -lt (nhỏ hơn), -ge (lớn hơn hoặc bằng), -le (nhỏ hơn hoặc bằng) kết hợp với cấu trúc rẽ nhánh if-elif-else để giới hạn tuổi trong khoảng 0-100 và phân loại in ra các chuỗi "đứa trẻ", "thiếu niên", hay "người lớn".
- **Phân quyền và thực thi:** Cấp quyền thực thi bằng lệnh chmod +x và khởi chạy file kịch bản check_age.sh trên Terminal.

*Minh chứng và giải thích:

```
check_age.sh X
1 #!/bin/bash
2 echo "Hay nhap tuoi cua ban:"
3 read age
4
5 if [ $age -lt 0 ] || [ $age -gt 100 ]; then
6     echo "Tuoi khong hop le. Phai nam trong khoang tu 0 den 100."
7     exit 1
8 fi
9
10 if [ $age -lt 12 ]; then
11     echo "Ban la mot dua tre"
12 elif [ $age -ge 12 ] && [ $age -le 18 ]; then
13     echo "Ban la mot thieu nien"
14 elif [ $age -gt 18 ]; then
15     echo "Ban la nguoi lon"
16 fi

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

$ chmod +x check_age.sh
$ ./check_age.sh
Hay nhap tuoi cua ban:
8
Ban la mot dua tre
$ ./check_age.sh
Hay nhap tuoi cua ban:
16
Ban la mot thieu nien
$ ./check_age.sh
Hay nhap tuoi cua ban:
24
Ban la nguoi lon
$
```

Thực thi file kịch bản và nhập các độ tuổi khác nhau để kiểm tra phân nhánh

Giải thích: Áp dụng các toán tử so sánh toán học (-lt, -gt, -le, -ge) cho biến số để đảm bảo logic rẽ nhánh đúng với yêu cầu đề bài. Cú pháp đa điều kiện && (AND) và || (OR) đóng vai trò thiết lập khoảng giá trị một cách chặt chẽ. Cuối cùng, kết quả thực thi được in ra tương ứng với điều kiện rẽ nhánh một cách chính xác.

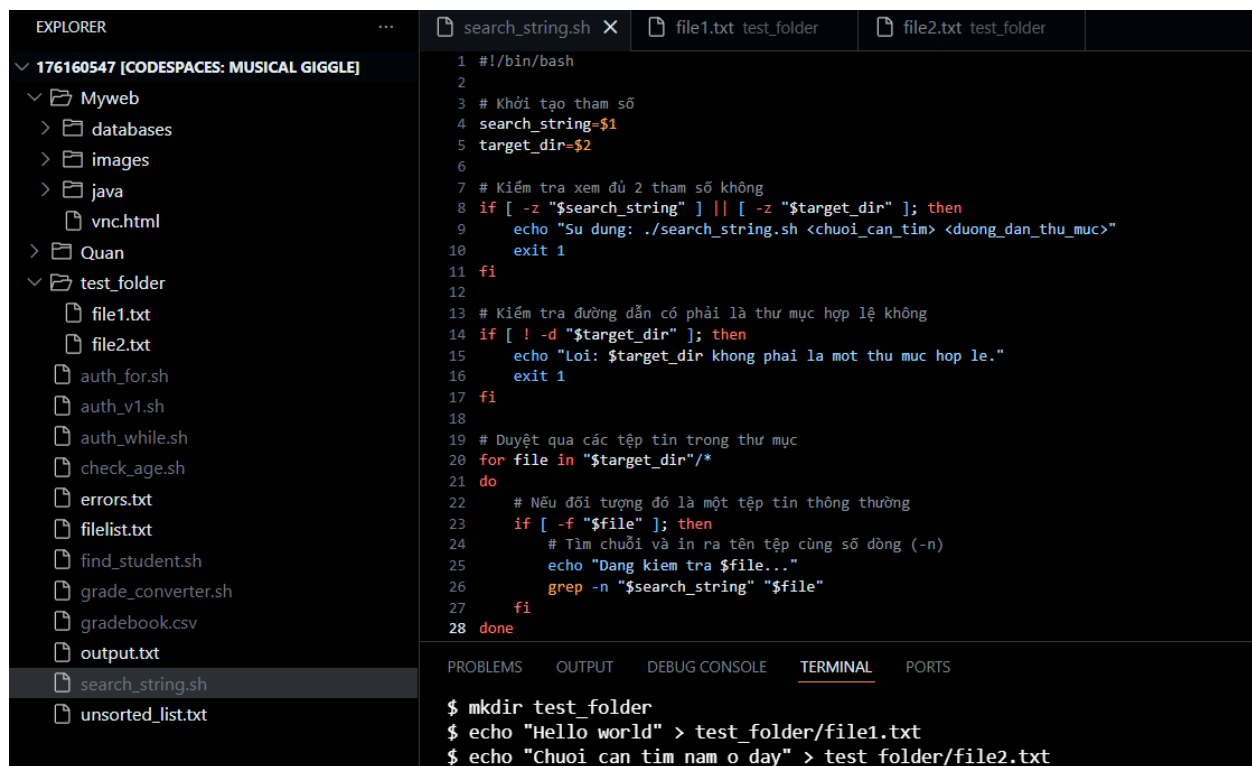
2.7 HOMEWORK 07:

*Phương pháp thực hiện:

- **Chuẩn bị dữ liệu mẫu:** Sử dụng các lệnh cơ bản (mkdir, echo, >) để tạo lập một thư mục tạm nghiệm chứa nhiều file .txt có nội dung khác nhau nhằm làm dữ liệu đầu vào để test.

- **Khởi tạo tệp kịch bản:** Viết script `search_string.sh` theo yêu cầu đề bài.
- **Tiếp nhận tham số:** Lấy 2 biến tham số `$1` (đại diện cho chuỗi cần tìm) và `$2` (đại diện cho đường dẫn thư mục đích). Đồng thời, dùng cấu trúc `if [ -z "$1" ]` để nhắc nhở nếu người dùng quên truyền tham số.
- **Kiểm tra đường dẫn:** Dùng `[ -d directory ]` để xác nhận thư mục do người dùng truyền vào có thực sự tồn tại hay không trước khi tiến hành quét file.
- **Quét và tìm kiếm:** Áp dụng vòng lặp `for file in "$2"/*` để duyệt mọi đối tượng trong thư mục. Lọc riêng các file văn bản thông thường bằng `[ -f file ]`. Cuối cùng áp dụng lệnh `grep -n` để tìm và in số dòng chứa từ khóa.
- **Cấp quyền và chạy:** Dùng lệnh `chmod +x` để cấp quyền cho tệp script và chạy thử với thư mục đã tạo ở bước 1.

*Minh chứng và giải thích:



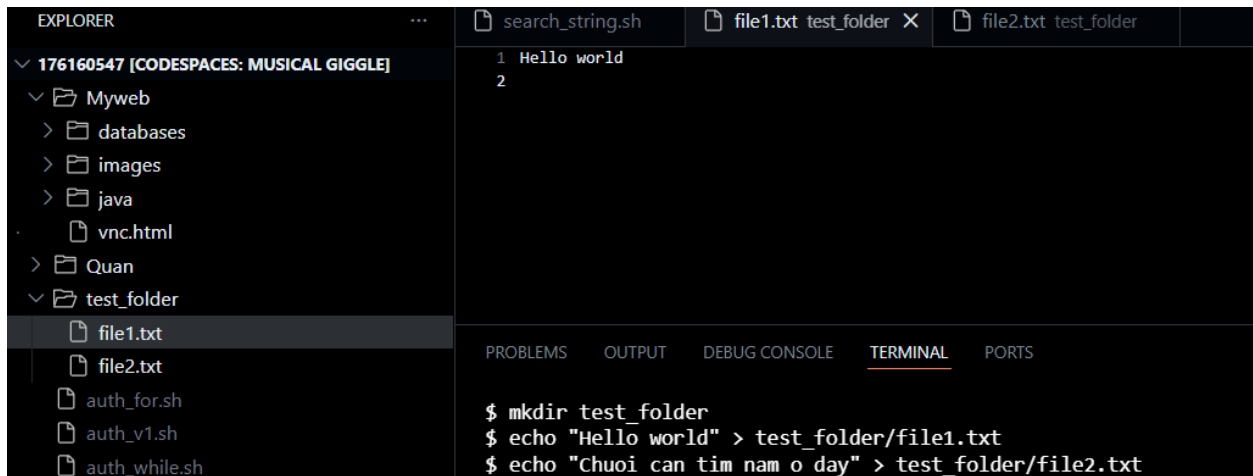
The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a directory structure with a 'test_folder' containing 'file1.txt' and 'file2.txt'. The code editor shows the script `search_string.sh` with the following content:

```
1 #!/bin/bash
2
3 # Khởi tạo tham số
4 search_string-$1
5 target_dir=$2
6
7 # Kiểm tra xem đủ 2 tham số không
8 if [ -z "$search_string" ] || [ -z "$target_dir" ]; then
9     echo "Sử dụng: ./search_string.sh <chuoi_can_tim> <duong_dan_thu_muc>"
10    exit 1
11 fi
12
13 # Kiểm tra đường dẫn có phải là thư mục hợp lệ không
14 if [ ! -d "$target_dir" ]; then
15     echo "Lỗi: $target_dir không phải là một thư mục hợp lệ."
16     exit 1
17 fi
18
19 # Duyệt qua các tệp tin trong thư mục
20 for file in "$target_dir"/*
21 do
22     # Nếu đối tượng đó là một tệp tin thông thường
23     if [ -f "$file" ]; then
24         # Tìm chuỗi và in ra tên tệp cùng số dòng (-n)
25         echo "Đang kiểm tra $file..."
26         grep -n "$search_string" "$file"
27     fi
28 done
```

The terminal output shows the execution of the script:

```
$ mkdir test_folder
$ echo "Hello world" > test_folder/file1.txt
$ echo "Chuoi can tim nam o day" > test_folder/file2.txt
```

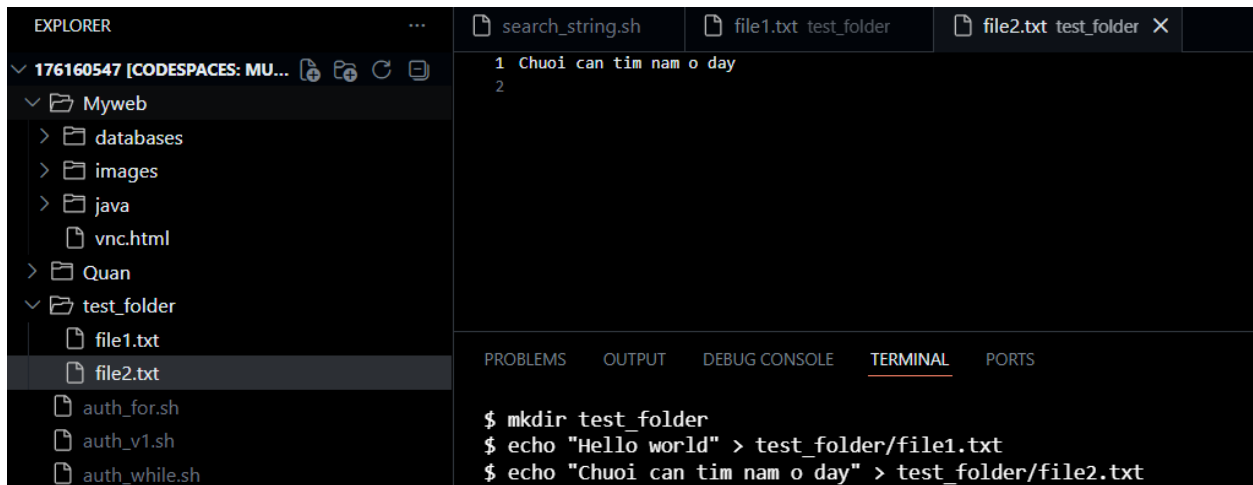
Thực thi chạy thử nghiệm kịch bản - Nội dung search_string.sh (1)



The screenshot shows the VS Code Explorer on the left with a project structure including folders like 'Myweb', 'databases', 'images', 'java', and 'test_folder'. The 'test_folder' is expanded, showing 'file1.txt' and 'file2.txt'. The Terminal on the right shows the following commands and output:

```
$ mkdir test_folder
$ echo "Hello world" > test_folder/file1.txt
$ echo "Chuoi can tim nam o day" > test_folder/file2.txt
```

Thực thi chạy thử nghiệm kịch bản - Nội dung file1.txt (2)



The screenshot shows the VS Code Explorer with 'file2.txt' selected. The Terminal shows the same commands as before, but the output in the editor area shows the content of 'file1.txt' and 'file2.txt' being checked:

```
1 Chuoi can tim nam o day
2
```

Thực thi chạy thử nghiệm kịch bản - Nội dung file2.txt (3)

```
$ chmod +x search_string.sh
$ ./search_string.sh "Chuoi can tim" test_folder
Dang kiem tra test_folder/file1.txt...
Dang kiem tra test_folder/file2.txt...
1:Chuoi can tim nam o day
$
```

Thực thi chạy thử nghiệm kịch bản – Kết quả (4)

- **\$1 và \$2:** Đây là các biến tham số dòng lệnh. Khi chạy lệnh `./search_string.sh "Chuoi can tim" test_folder`, "Chuoi can tim" sẽ được nạp vào \$1 và test_folder được nạp vào \$2.
- **[! -d "\$target_dir"]:** Toán tử -d trả về true nếu đường dẫn là một thư mục (directory). Dấu chấm than ! dùng làm toán tử phủ định (NOT), có nghĩa là: "Nếu đường dẫn KHÔNG phải là thư mục thì báo lỗi".

- **for file in "\$target_dir"/*:** Vòng lặp for có cấu trúc duyệt qua các phần tử thông qua ký tự đại diện *. Khối lệnh này sẽ lấy tất cả các mục có trong target_dir gán lần lượt vào biến file để xử lý.
- **[-f "\$file"]:** Toán tử -f trả về true nếu biến file đang mang giá trị là một tệp tin thông thường (file), giúp lọc bỏ các mục là thư mục con (folder) khỏi lệnh tìm kiếm.
- **grep -n:** Lệnh grep dùng để lọc và tìm kiếm chuỗi trong văn bản. Tham số -n giúp đánh số thứ tự dòng chứa chuỗi để đáp ứng yêu cầu in ra dòng chứa chuỗi của đề bài.

3. BONUS:

3.1 BONUS 01:

*Phương pháp thực hiện:

- **Chuẩn bị dữ liệu:** Cài đặt công cụ git bằng lệnh `sudo apt-get install git` (nếu chưa có). Sau đó, sử dụng lệnh `git clone` để tải toàn bộ dữ liệu từ kho lưu trữ GitHub về máy ảo theo đường dẫn mà đề bài cung cấp.

- **Khởi tạo kịch bản:** Tạo tệp `sort_images.sh`.

- **Kiểm tra và tạo thư mục:** Sử dụng cấu trúc rẽ nhánh `if` kết hợp kiểm tra thư mục `[ ! -d ]` (với `!` là toán tử NOT phủ định) để kiểm tra xem hai thư mục PNG và JPG đã tồn tại chưa. Nếu chưa thì dùng lệnh `mkdir` để tạo mới.

- **Cấp quyền và chạy:** Dùng lệnh `chmod +x` để cấp quyền cho tệp script và chạy thử.

- **Đếm và di chuyển tệp:**

- Dùng ký tự điều khiển quá trình gộp lệnh `$(command)` để thực thi lệnh đếm số lượng file (`ls | wc -l`) và gán ngay kết quả vào một biến đếm.
- Kết hợp lệnh `mv` để di chuyển các tệp có đuôi `.png` và `.jpg` từ thư mục tải về sang hai thư mục đích tương ứng, đồng thời in kết quả đếm ra màn hình.

***Minh chứng và giải thích:**

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

/workspaces/ $ sudo apt-get install git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
The following package was automatically installed and is no longer required:
  libwayland-server0
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
/workspaces/ $ git clone https://github.com/locth/OS_LAB2_IMG.git
Cloning into 'OS_LAB2_IMG'...
remote: Enumerating objects: 98, done.
remote: Total 98 (delta 0), reused 0 (delta 0), pack-reused 98 (from 1)
Receiving objects: 100% (98/98), 40.60 MiB | 14.58 MiB/s, done.
/workspaces/ $ ls
176160547/  OS_LAB2_IMG/
/workspaces/ $
```

Thực hiện clone thư mục image từ link theo yêu cầu đề bài

```
GNU nano 7.2                                sort_images.sh *
#!/bin/bash

# a. Kiểm tra và tạo thư mục PNG, JPG
if [ ! -d "PNG" ]; then
    mkdir PNG
fi

if [ ! -d "JPG" ]; then
    mkdir JPG
fi

# b. Đếm, di chuyển và in số lượng file PNG
# Lọc luồng lỗi (2>/dev/null) để tránh báo lỗi nếu không có file nào
png_count=$(ls OS_LAB2_IMG/*.png 2>/dev/null | wc -l)
if [ $png_count -gt 0 ]; then
    mv OS_LAB2_IMG/*.png PNG/
fi
echo "Số lượng hình ảnh PNG: $png_count"

# c. Đếm, di chuyển và in số lượng file JPG
jpg_count=$(ls OS_LAB2_IMG/*.jpg 2>/dev/null | wc -l)
if [ $jpg_count -gt 0 ]; then
    mv OS_LAB2_IMG/*.jpg JPG/
fi
echo "Số lượng hình ảnh JPG: $jpg_count"

```

^G Help ^O Write Out ^W Where Is ^X Cut ^T Execute ^G Location ^U Undo ^A Set Mark ^_ To Bracket
^X Exit ^R Read File ^M Replace ^U Paste ^J Justify ^_ Go To Line ^E Redo ^C Copy ^O Where Was

Sử dụng lệnh nano để tạo tệp sort_images.sh


```
/workspaces/ $ chmod +x sort_images.sh
/workspaces/ $ ls
176160547/ OS LAB2 IMG/ sort_images.sh*
/workspaces/ $ ./sort_images.sh
Số lượng hình ảnh PNG: 18
Số lượng hình ảnh JPG: 60
/workspaces/ $ ls
176160547/ JPG/ OS LAB2 IMG/ PNG/ sort_images.sh*
/workspaces/ $ ls PNG/
41pK613-png-wallpapers.png gybTsp2-png-wallpapers.png kxzKRVe-png-wallpapers.png Qsbbfie-png-wallpapers.png w6idpDt-png-wallpapers.png
dLZ5oMU-png-wallpapers.png IdKBq80-png-wallpapers.png LzJoPy4-png-wallpapers.png rhS7Qoe-png-wallpapers.png Z8guuoT-png-wallpapers.png
do4xlab-png-wallpapers.png kaPwu0d-png-wallpapers.png Nmk93LV-png-wallpapers.png UBQjv5C-png-wallpapers.png
EQdA2ld-png-wallpapers.png Ku4XHvm-png-wallpapers.png qjco8MH-png-wallpapers.png w1Dgtgd-png-wallpapers.png
/workspaces/ $ ls JPG/
2016-01-10_NorthAvocaBeach_EN-AU12399626703_1920x1080.jpg
2016-01-13_MouflonsForest_ROW13263118682_1920x1080.jpg
2016-01-14_Florianopolis_PT-BR8264316884_1920x1080.jpg
2016-01-19_KakhetiGeorgia_ROW9666115250_1920x1080.jpg
2016-01-23_FjordPano_JA-JP9603983993_1920x1080.jpg
2016-01-25_Bibury_EN-GB7595942029_1920x1080.jpg
2016-01-25_IndiaRepublicDay_EN-IN10578454849_1920x1080.jpg
2016-01-26_AyersRock_ROW12130770257_1920x1080.jpg
2016-01-27_LeafTailGecko_ROW12526482913_1920x1080.jpg
2016-01-28_CurbarEdge_EN-GB8017846533_1920x1080.jpg
2016-01-29_BigGardenBirdwatch_EN-GB14409615569_1920x1080.jpg
2016-01-30_HammockCypress_ROW11689927667_1920x1080.jpg
2016-01-31_FarmSnowVillage_ZH-CN11121852629_1920x1080.jpg
2016-01-31_H3Interstate_ROW9683976880_1920x1080.jpg
2016-01-31_ZecheZollverein_DE-DE6956418086_1920x1080.jpg
amanitas_mushrooms_autumn_129262_4950x7421.jpg
clouds_porous_sky_125791_6000x4000.jpg
clouds_sky_porous_103701_2560x1600.jpg
clouds_sky_porous_123908_5963x2912.jpg
clouds_sky_porous_129343_3024x4032.jpg
desert_canyon_height_sky_4928_2560x1600.jpg
desert_sand_stones_104109_1920x1080.jpg
desert_water_grass_sand_sky_92737_1920x1200.jpg
flowers_branches_blossom_117474_6016x4000.jpg
flowers_branches_buds_leaves_115624_4147x3110.jpg
flowers_branches_leaves_smeared_glare_43392_1920x1200.jpg
```

Quá trình thực thi file `sort_images.sh` và kết quả

- **[! -d "PNG"]**: `-d` là toán tử kiểm tra thư mục. Khi kết hợp với toán tử NOT (!), biểu thức này mang ý nghĩa "Nếu thư mục PNG chưa tồn tại" thì mới thực thi khối lệnh tạo thư mục bên trong. Ý nghĩa tương tự đối với file JPG.
- **png_count=\$(ls OS_LAB2_IMG/*.png | wc -l)**: Cấu trúc `$(...)` dùng để lấy kết quả đầu ra của các lệnh bên trong nó và gán trực tiếp làm giá trị cho biến `png_count`. Trong đó, `ls OS_LAB2_IMG/*.png` sẽ liệt kê các file PNG, và pipeline `|` sẽ đẩy danh sách này cho lệnh `wc -l` đếm số lượng dòng tương ứng với số lượng file. Ý nghĩa tương tự đối với file JPG.
- **mv OS_LAB2_IMG/*.png PNG/**: Lệnh di chuyển toàn bộ các file `.png` (sử dụng wildcard `*` đại diện cho mọi tên file) vào trong thư mục đích PNG vừa tạo. Ý nghĩa tương tự đối với file JPG.

3.2 BONUS 02:

*Phương pháp thực hiện:

- **Tạo tệp đầu vào**: Dùng lệnh `echo` kết hợp chuyển hướng luồng (`>`) hoặc trình soạn thảo nano để tạo tệp văn bản `subject.txt` chứa danh sách mã môn học (mỗi mã 1 dòng).

- **Viết kịch bản nhập liệu:** Tạo file create_subjects.sh. Sử dụng lệnh read để yêu cầu người dùng nhập họ và tên.
- **Quản lý khoảng trắng:** Sử dụng dấu ngoặc kép " " bao quanh biến \$fullname trong lệnh mkdir "\$fullname" để giữ nguyên chuỗi khoảng trắng, đảm bảo hệ thống tạo ra một thư mục duy nhất thay vì tạo nhiều thư mục rời rạc.
- **Đọc tệp và tạo thư mục con:** Áp dụng vòng lặp while kết hợp với biến môi trường phân tách IFS= để đọc từng dòng của tệp subject.txt. Với mỗi mã môn học trích xuất được, dùng mkdir để tạo thư mục con bên trong thư mục họ tên cha.
- **Cấp quyền và thực thi:** Dùng lệnh chmod +x để cấp quyền cho tệp script và chạy thử.

***Minh chứng và giải thích:**



```
subject.txt X
1 CE103
2 CE124
3 CE213
4 IT007
5 PE231
6 SS008
7 SS010
8

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
$ cat <<EOF > subject.txt
> CE103
> CE124
> CE213
> IT007
> PE231
> SS008
> SS010
> EOF
$
```

Thực hiện tạo file subject.txt

```
create_subjects.sh X
1 #!/bin/bash
2
3 # a. Yêu cầu nhập họ tên và tạo thư mục gốc
4 echo "Vui long nhap ho va ten cua ban:"
5 read fullname
6
7 # Sử dụng dấu ngoặc kép để tạo đúng 1 thư mục có khoảng trắng
8 mkdir "$fullname"
9
10 # b. Đọc tệp subject.txt và tạo thư mục con
11 while IFS= read -r subject_code
12 do
13     # Bỏ qua các dòng rỗng (nếu có)
14     if [ -n "$subject_code" ]; then
15         mkdir "$fullname/$subject_code"
16     fi
17 done < "subject.txt"
18
19 echo "Da tao thanh cong danh sach mon hoc trong thu muc: $fullname"
20
```

Nội dung file kịch bản create_subjects.sh

```
$ chmod +x create_subjects.sh
$ ./create_subjects.sh
Vui long nhap ho va ten cua ban:
Nguyen Dong Quan
Da tao thanh cong danh sach mon hoc trong thu muc: Nguyen Dong Quan
$ ls
auth_for.sh*  check_age.sh*  filelist.txt  grade_converter.sh*  output.txt  subject.txt
auth_v1.sh*   create_subjects.sh*  find_student.sh*  Myweb/  Quan/  test_folder/
auth_while.sh*  errors.txt  gradebook.csv  'Nguyen Dong Quan/'  search_string.sh*  unsorted_list.txt
$ ls -R "Nguyen Dong Quan"
'Nguyen Dong Quan':
CE103/  CE124/  CE213/  IT007/  PE231/  SS008/  SS010/

'Nguyen Dong Quan/CE103':
'Nguyen Dong Quan/CE124':
'Nguyen Dong Quan/CE213':
'Nguyen Dong Quan/IT007':
'Nguyen Dong Quan/PE231':
'Nguyen Dong Quan/SS008':
'Nguyen Dong Quan/SS010':
$
```

Cấp quyền, thực thi file kịch bản và xem kết quả

- **mkdir "\$fullname"**: Dấu ngoặc kép (" ") rất quan trọng trong tình huống này. Nó giúp "giữ nguyên giá trị của hầu hết các ký tự" (bao gồm dấu cách). Nếu bỏ ngoặc kép (chỉ gõ `mkdir $fullname`), khi bạn nhập *Nguyen Dong Quan*, lệnh sẽ hiểu lầm là tạo 3 thư mục riêng biệt: *Nguyen*, *Dong* và *Quan*. Việc thêm ngoặc kép đảm bảo tạo ra một thư mục duy nhất mang tên *Nguyen Dong Quan*.
- **while IFS= read -r subject_code**: Khối vòng lặp while sử dụng biến môi trường IFS (Internal Field Separator) rỗng để đảm bảo lệnh read lấy trọn vẹn nguyên một dòng văn bản từ tệp `subject.txt` gán vào biến `subject_code`.
- **mkdir "\$fullname/\$subject_code"**: Câu lệnh này tận dụng đường dẫn tương đối. Nó truy cập vào thư mục cha đã tạo ở bước a và tiếp tục tạo ra thư mục con mang tên mã môn học bên trong đó. Ký hiệu `<"subject.txt"` ở cuối vòng lặp while có vai trò chuyển hướng luồng dữ liệu, liên tục đẩy nội dung file văn bản vào vòng lặp cho đến khi hết.
- **Ý nghĩa**: Kịch bản giúp tự động hóa việc khởi tạo hàng loạt cây thư mục lồng nhau (thư mục môn học nằm trong thư mục sinh viên) từ tệp dữ liệu có sẵn, giúp tổ chức file khoa học và tiết kiệm thời gian.